«Общий знаменатель» — техдок для хакатона

Oct 25, 2026 · @Bodya

Обзор

Делаем командный чат с живой доской решения рядом. ИИ-агент читает переписку, раскладывает её на цели, предложения и ограничения, сам находит общее и конфликты и фиксирует согласованные решения с обоснованием.

Проблема: команда обсуждает задачу в десятках сообщений, а общего понимания и согласованного решения так и не появляется.

Ключевая фишка: агент не склеивает всё подряд. Он различает две ситуации:

- **одна цель, разные слова или способы** — агент предлагает общий вариант;
- **настоящий конфликт интересов** — агент не объединяет, а прямо спрашивает команду, что приоритетнее, и показывает, чем жертвует каждый вариант.

Подача: тема 3 «Второй мозг». Журнал решений с обоснованиями — это память команды, а вмешательства агента — применение контекста в нужный момент. Репозиторий открываем под MIT, чтобы зацепить и тему 4.

Объём MVP (обязательно к 15:30):

1. Чат на несколько участников в реальном времени.
2. Извлечение из сообщений: цели, предложения, ограничения, возражения, вопросы.
3. Связи «одна цель» и «конфликт» на доске.
4. Вмешательства агента трёх типов: объединение, вопрос о приоритете, анализ последствий.
5. Голосование за предложение агента и журнал решений с обоснованием.
6. Подсветка решений, которые надо пересмотреть после смены требования.

Если останется время: экспорт итогов в markdown, ручное редактирование карточек, эмбеддинги для дедупликации, упоминание агента через @.

Сценарий демо

Демо идёт вживую с 4 ноутбуков, каждый играет свою роль. На проекторе доска. За 8 сообщений жюри видит все три типа вмешательства. Этот же сценарий — первый тест в eval и кнопка «воспроизвести» на случай проблем.

#	Кто	Сообщение	Что делает агент	Что на доске
1	Руководитель	«Нужно увеличить продажи на 15% за квартал, большого бюджета нет»	Молчит	Цель G1 «+15% продаж», ограничение C1 «низкий бюджет»
2	Маркетолог	«Нужны персональные предложения, люди уходят без покупки»	Молчит	Предложение P1, скрытая цель «конверсия»
3	Дизайнер	«Давайте сократим путь до покупки, сейчас 5 шагов оформления»	Замечает общую цель, но ждёт ещё сообщение (cooldown)	P2, зелёное ребро «одна цель» P1–P2
4	Разработчик	«Сложную систему рекомендаций не потянем, нас двое и месяц»	**Объединение:** «Вы трое решаете одну проблему — люди уходят до покупки. Предлагаю: оформление в 2 шага + готовые наборы товаров по простым правилам. Согласны?»	Ограничение C2 «2 разработчика, месяц», карточка предложения с кнопками ✅/❌
5	Все	жмут ✅	Фиксирует решение	Решение D1 с обоснованием: учитывает P1 (частично), P2, C1, C2

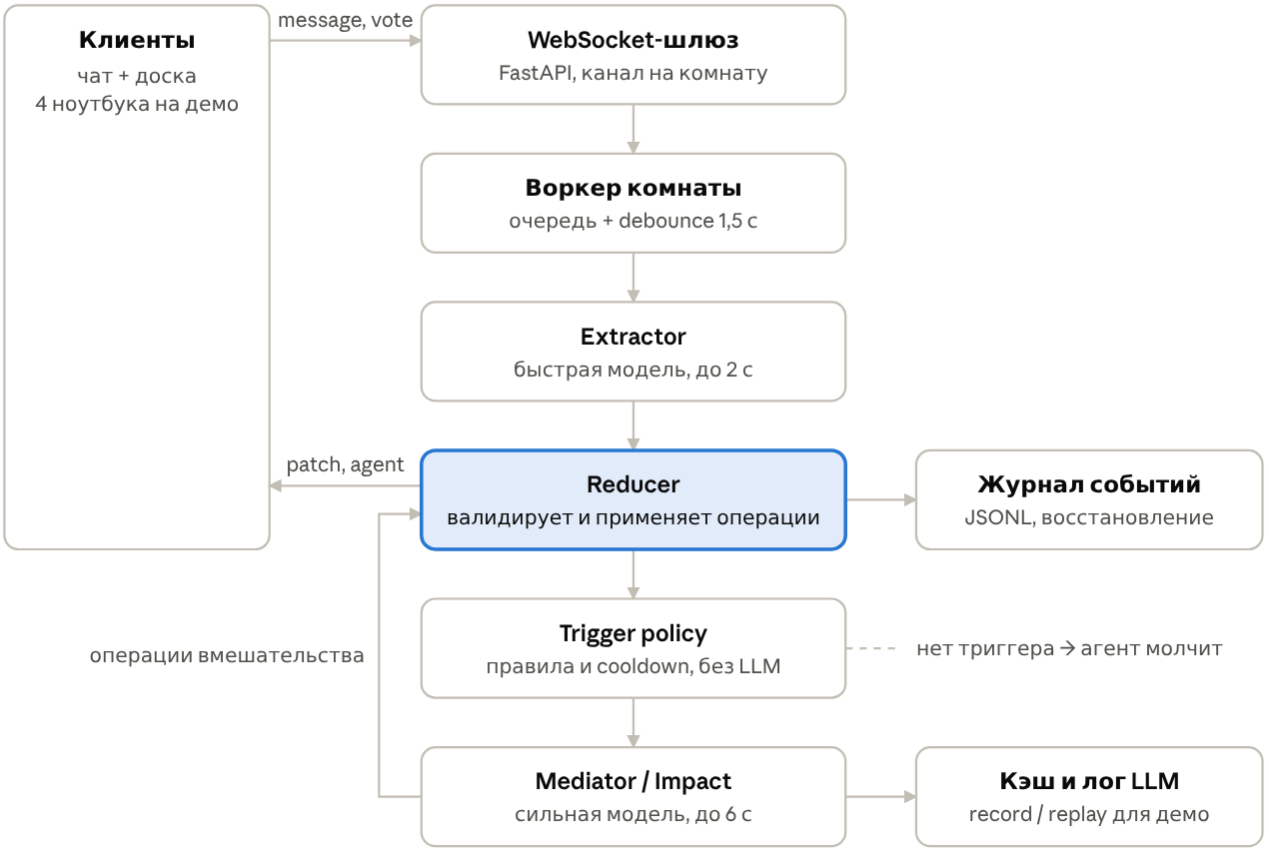
#	Кто	Сообщение	Что делает агент	Что на доске
6	Маркетолог	«И ещё нужна рассылка со скидкой 20% всем, чтобы вернуть старых клиентов»	**Конфликт:** «Тут настоящий конфликт: скидка 20% бьёт по марже, а бюджет ограничен. Что важнее: вернуть клиентов или сохранить маржу?» Не склеивает	P3, красное ребро «конфликт» P3–C1, открытый вопрос Q1
7	Руководитель	«Маржа важнее. Скидка максимум 5% и только на наборы»	Фиксирует приоритет	Q1 закрыт, решение D2 «скидка 5% на наборы», зависит от C1
8	Руководитель	«Новости: бюджет дали, но срок — 2 недели вместо месяца»	**Последствия:** «Изменились C1 и C2. D2 держалось на нехватке бюджета — скидку можно пересмотреть. D1 за 2 недели под угрозой: начать только с оформления, наборы потом?»	D1 и D2 краснеют (статус needs_review)

На слайде после демо показываем журнал решений: что выбрали, чьи пожелания учли, чем пожертвовали и почему.

Архитектура

Один Python-сервис на FastAPI держит WebSocket, состояние комнат и ML-пайплайн. Фронт на React только рисует. Главный принцип: **LLM никогда не переписывает состояние целиком**. Она возвращает список операций, а детерминированный редьюсер на Python проверяет и применяет их. Так модель не может молча потерять карточку, токенов уходит меньше, а фронт получает готовые диффы для анимации.

LLM предлагает операции, состояние меняет только редьюсер



архитектура · путь сообщения от чата до доски

Всё, что предлагают модели, проходит через Reducer, и только он рассылает изменения клиентам. Агент говорит, лишь когда сработал триггер.

Слой	Выбор	Почему
Бэкенд	Python 3.11+, FastAPI, uvicorn	Родной для ML, WebSocket из коробки, async
Схемы и валидация	Pydantic v2	Одни и те же модели — и JSON-схема для LLM, и валидация ответа
LLM-клиент	openai SDK с base_url на OpenRouter (или прямой провайдер)	Модели меняются одной строкой конфига
Хранение	Состояние в памяти + журнал событий в JSONL по комнате	Ноль инфраструктуры; состояние восстанавливается проигрыванием событий

Слой	Выбор	Почему
Кэш LLM	Файловый кэш по хэшу запроса	Демо воспроизводимо и работает без сети (раздел «Модели»)
Фронт	React + Vite, React Flow для доски, zustand для стора	Доска с узлами и рёбрами без своего движка
Деплой	Один ноутбук в локальной сети или Railway/Render	Для демо хватит одного процесса; заранее проверьте, что университетский Wi-Fi пропускает WebSocket

Путь одного сообщения:

1. Клиент шлёт `message` по WebSocket. Сервер сразу рассылает его всем и кладёт в очередь комнаты.
2. Воркер комнаты ждёт 1,5 с (debounce) и забирает пачку новых сообщений. По комнате всегда работает один обработчик, гонок нет.
3. **Extractor** (быстрая модель) возвращает тип сообщения и операции над состоянием.
4. **Reducer** валидирует операции и применяет их. Версия состояния +1, всем уходит `patch`.
5. **Trigger policy** (правила, без LLM) решает, нужно ли агенту говорить.
6. Если да, **Mediator** (сильная модель) пишет вмешательство. Пока он думает, всем уходит `agent_status: thinking`.
7. Reducer применяет операции вмешательства (новое предложение, вопрос). В чат и на доску уходят `agent` и `patch`.
8. Всё пишется в журнал событий и в лог LLM-вызовов.

Модель данных

Всё состояние комнаты — один объект `RoomState` с версией. Карточки всех типов живут в одном списке `items`: так проще ссылаться по id и рисовать граф. Короткие читаемые id (`P3`, `C1`, `D2`) выдаёт сервер, а не LLM: модель только ссылается на уже существующие id или просит создать новую карточку.

```
# models.py
from typing import Literal
from pydantic import BaseModel, Field
```

```
ItemKind = Literal["proposal", "constraint", "objection", "question"]

class Goal(BaseModel):            # канонический словарь целей
    id: str                       # G1, G2...
    text: str                     # "повысить конверсию в покупку"
    is_business_goal: bool = False  # главная цель задачи

class Item(BaseModel):
    id: str                       # P1, C1, O1, Q1
    kind: ItemKind
    author: str                   # имя или роль
    text: str                     # короткая нормализованная формулировка
    goal_id: str | None = None    # зачем человек этого хочет -> Goal.id
    means: str | None = None      # как именно предлагает достичь
    status: Literal["active", "merged", "resolved", "dropped"] = "active"
    source_msgs: list[int] = []   # из каких сообщений вытащено
    version: int = 1

class Relation(BaseModel):
    id: str                       # R1...
    a: str; b: str                # id карточек
    type: Literal["same_goal", "complementary", "conflict", "duplicate"]
    explanation: str              # одно предложение для тултипа на ребре
    confidence: float = Field(ge=0, le=1)
    status: Literal["suggested", "confirmed", "dismissed"] = "suggested"

class Decision(BaseModel):
    id: str                       # D1...
    text: str
    rationale: str                # почему выбрали
    covers: dict[str, str]        # item_id -> как учтено
    sacrifices: dict[str, str]    # item_id -> чем пожертвовали и почему
    depends_on: list[str]         # id ограничений и карточек, на которых
держится
    status: Literal["proposed", "accepted", "needs_review", "superseded"] =
"proposed"
    votes: dict[str, bool] = {}   # user -> за/против
    history: list[str] = []       # предыдущие формулировки и причины смены

class Message(BaseModel):
    seq: int; author: str; role: str; text: str; ts: float
    is_agent: bool = False

class RoomState(BaseModel):
    room_id: str
    version: int = 0
```

```
    goals: list[Goal] = []
    items: list[Item] = []
    relations: list[Relation] = []
    decisions: list[Decision] = []
    participants: dict[str, str] = {}   # user -> role
    counters: dict[str, int] = {}       # для выдачи id по префиксу
```

Операции (patch ops) — закрытый набор. Их возвращает LLM и применяет reducer. Операция со ссылкой на несуществующий id отбрасывается и логируется, остальные применяются.

Операция	Поля	Кто вызывает
`add_goal`	text, is_business_goal	Extractor
`add_item`	kind, text, goal_id или new_goal, means	Extractor
`attach_to_item`	item_id — сообщение повторяет уже сказанное	Extractor
`update_item`	item_id, text и/или means — человек уточнил своё	Extractor
`change_constraint`	item_id, new_text — ограничение изменилось или снято	Extractor
`add_relation`	a, b, type, explanation, confidence	Extractor (кандидат), Mediator (вердикт)
`propose_decision`	text, rationale, covers, sacrifices, depends_on	Mediator
`add_question`	text, options, related_ids	Mediator
`resolve_question`	question_id, answer	Extractor
`cast_vote`	decision_id, user, value	Клиент (без LLM)
`mark_needs_review`	decision_id, reason	Impact-анализ (без LLM)

Журнал событий `data/{room_id}.jsonl` хранит сообщения, применённые операции и вмешательства. При рестарте сервера состояние собирается проигрыванием журнала без LLM.

ML-пайплайн, шаг 1: Extractor

Extractor превращает сырое сообщение в операции над состоянием. Работает на каждую пачку сообщений, поэтому нужна быстрая дешёвая модель. Цель по задержке — до 2 с.

Вход:

- новые сообщения (автор, роль, текст);
- последние 8 сообщений для контекста («да, согласен с ним» без них не понять);
- компактное состояние: словарь целей и активные карточки одной строкой (`P1 [маркетолог] персональные предложения → G2`), открытые вопросы, принятые решения.

Выход: `message_type` для каждого сообщения и список `ops`. Типы сообщений: idea, requirement, objection, change, agreement, question, answer, chatter. На chatter («ок», «го обедать») операций нет.

Канонизация целей — главный ML-трюк

У каждого предложения два слоя: **цель** (зачем) и **средство** (как). Extractor обязан либо привязать предложение к цели из словаря, либо создать новую, если ни одна не подходит. В промпте явно просим переиспользовать существующие цели, если смысл тот же.

Это даёт дешёвый детерминированный сигнал: **два предложения разных авторов с одним `goal_id` и разными `means`** — кандидат в same_goal. На стороне Python это один groupby, без лишнего вызова LLM. В демо «персональные предложения» и «короткий путь до покупки» оба уходят в G2 «повысить конверсию».

Дедупликация

- Если сообщение повторяет уже существующую карточку другими словами, нужен `attach_to_item`, а не новая карточка.
- Если повторяет чужую карточку — `attach_to_item` + автор добавляется как поддержавший (для доски это счётчик «+1»).
- **После MVP:** считать эмбеддинг каждой карточки (multilingual-e5 локально или embeddings API). Пары с косинусом выше порога передавать в промпт как подсказку «возможный дубль». Порог подобрать на eval-наборе. При десятках карточек LLM справляется и без этого, но для питча про масштаб пригодится.

Кандидаты в конфликт

Extractor только ставит флаг `possible_conflict_with: [ids]`, если новая карточка на вид противоречит другой или ограничению. Вердикт «настоящий конфликт или нет» выносит Mediator на сильной модели. Быстрая модель чаще ошибается именно в этом различении, а оно — суть продукта.

Правила редьюсера после Extractor

1. Валидировать JSON по Pydantic-схеме. При ошибке — один ретрай с текстом ошибки, потом пропуск с записью в лог.
2. Отбросить операции с несуществующими id.
3. Выдать id новым карточкам и целям (счётчик по префиксу).
4. Посчитать кандидатов same_goal (groupby по `goal_id`) и собрать флаги конфликтов — это вход для trigger policy.
5. Если в операциях есть `change_constraint`, запустить impact-анализ (раздел «Согласование и анализ последствий»).

ML-пайплайн, шаг 2: Trigger policy и Mediator

Когда агенту говорить, решают правила на Python. Что именно сказать, решает сильная модель. Так агент не встревает на каждое сообщение, а его поведение на демо предсказуемо.

Trigger policy

Триггер	Условие	Приоритет	Cooldown
`IMPACT`	`change_constraint`, и от этого ограничения зависит хотя бы одно принятое решение	1	Нет, сразу
`MENTION`	В сообщении есть @агент	1	Нет
`CONFLICT`	У новой карточки есть `possible_conflict_with`	2	1 сообщение
`MERGE`	Есть ⩾ 2 активных предложения разных авторов с одним `goal_id`, и по ним ещё не было вмешательства	3	3 сообщения
`NUDGE` (опционально)	Открытый вопрос висит > 6 сообщений	4	6 сообщений

За один цикл срабатывает один триггер с наивысшим приоритетом. Остальные ждут следующего. Cooldown считается в сообщениях людей с последнего вмешательства. Пороги вынесите в конфиг, чтобы подкручивать под сценарий демо.

Таксономия отношений — сердце продукта

Mediator относит каждую пару (или группу) карточек к одному из четырёх классов. От класса зависит действие.

Класс	Признак	Пример	Действие агента
`same_goal`	Одна цель, разные средства; можно совместить	«Персональные предложения» + «короткий путь до покупки»	Синтез общего предложения
`complementary`	Разные цели, но обе достижимы вместе в рамках ограничений	«Данные для CRM» + «без обязательной регистрации» → гостевой заказ и регистрация после покупки	Компромисс, задевающий обе цели

Класс	Признак	Пример	Действие агента
`conflict`	Цели конкурируют за один ресурс, одновременно не достижимы	«Скидка 20% всем» vs «маржа, низкий бюджет»	**Не склеивать.** Вопрос о приоритете с вариантами и ценой каждого
`misunderstanding`	Кажется, что спорят, но говорят о разном или одном и том же разными словами	«Нужен лендинг» vs «нет, нужна простая главная»	Уточняющий вопрос или слияние карточек

Процедура рассуждения внутри Mediator

Модель сначала заполняет поля рассуждения и только потом вердикт. Порядок полей в JSON-схеме задаёт порядок мышления:

1. `goals`: для каждой карточки — что человек на самом деле хочет получить.
2. `same_underlying_goal`: true/false и почему.
3. `jointly_satisfiable`: можно ли удовлетворить все цели сразу при текущих ограничениях. Если нет — какой ресурс они делят (деньги, время, маржа, внимание пользователя).
4. `relation_type` и `confidence`.
5. `intervention`: текст для чата + операции (`propose_decision` или `add_question`).

Защита от принудительного слияния

Эти правила проверяет Python после ответа модели, а не сама модель:

- `jointly_satisfiable = false` → `propose_decision` запрещён, разрешён только `add_question`. Если модель всё же предложила решение — ретрай с замечанием.
- `confidence < 0.6` → вместо вердикта уточняющий вопрос к авторам.
- `propose_decision` обязан содержать непустой `covers`. Каждая карточка из триггера должна попасть либо в `covers`, либо в `sacrifices` с объяснением. Молча потерять чьё-то пожелание нельзя.
- Все id в ответе должны существовать в состоянии.

На питче это звучит так: «Агент физически не может склеить конфликт и не может потерять чьё-то предложение — это гарантирует код, а не промпт».

Стиль вмешательства

Не длиннее 3 предложений. Структура: что заметил → что предлагает или спрашивает → чьи пожелания учтены. К людям обращается по ролям, без канцелярита и вводных «отличная идея!».

Согласование и анализ последствий

Обе части почти целиком детерминированные. LLM только пишет текст объяснения. Именно поэтому подсветка на демо сработает надёжно.

Жизненный цикл решения

1. Mediator создаёт `propose_decision` → статус `proposed`. В чате появляется карточка с кнопками ✅/❌ и списком «учитывает / жертвует».
2. Участники голосуют (`cast_vote`). Голос можно поменять.
3. Все присутствующие нажали ✅ → `accepted`. Решение уходит в журнал, а карточки из `covers` получают статус `resolved`.
4. Кто-то нажал ❌ → поле для причины. Причина идёт в пайплайн как сообщение типа objection. Mediator пересобирает предложение, старая формулировка уходит в `history`.
5. Ответ на вопрос о приоритете (`resolve_question`) запускает Mediator ещё раз: он оформляет решение, проигравшая сторона попадает в `sacrifices` с причиной.

Для демо правило «все присутствующие за». В конфиге можно переключить на «большинство и ни одного против».

Анализ последствий

У каждого решения есть `depends_on` — ограничения и карточки, на которых оно держится. Их заполняет Mediator при синтезе. Когда приходит `change_constraint`, обход графа зависимостей находит затронутые решения:

```
# impact.py
def affected_decisions(state: RoomState, changed_ids: set[str]) ->
list[Decision]:
    hit, frontier = set(), set(changed_ids)
    while frontier:
        nxt = set()
        for d in state.decisions:
            if d.status == "accepted" and d.id not in hit and frontier &
set(d.depends_on):
                hit.add(d.id)
                nxt.add(d.id)          # решения могут зависеть от решений
        frontier = nxt
    return [d for d in state.decisions if d.id in hit]
```

Дальше:

1. Все найденные решения сразу получают `needs_review`. Доска краснеет ещё до ответа LLM.
2. Impact-промпт получает старое и новое ограничение и затронутые решения. Для каждого решения он возвращает вердикт `keep` / `revise` / `drop`, причину и новую формулировку для `revise`.
3. `keep` → решение возвращается в `accepted` с пометкой в history. `revise` → новое предложение на голосование, старое получает `superseded`. `drop` → вопрос к команде.

Журнал решений — отдельная вкладка на доске. По каждому решению: текст, почему выбрали, чьи пожелания учли, чем пожертвовали, на чём держится, история версий. Это и есть «второй мозг» команды из темы 3.

Промпты

Промптов три: Extractor, Mediator, Impact. Храните их файлами в `prompts/` под git, а не строками в коде: так каждый прогон eval привязан к версии промпта. Ответ везде через structured output (JSON schema из Pydantic), температура 0–0,2.

Extractor — системный промпт

```
Ты — модуль разбора командного обсуждения бизнес-задачи. Твоя задача —
превратить
новые сообщения чата в операции над общей картой задачи. Ты НЕ отвечаешь
людям
и НЕ предлагаешь решений — только разкладываешь сказанное.
```

```
Карта задачи состоит из:
- ЦЕЛИ (G*): ЗАЧЕМ люди что-то предлагают. Коротко, 3–7 слов, без способа:
  "повысить конверсию в покупку", "сохранить маржу", "уложиться в срок".
  Одна цель помечена как главная бизнес-цель.
- ПРЕДЛОЖЕНИЯ (P*): что человек предлагает сделать. У каждого есть goal_id
(зачем)
  и means (как).
- ОГРАНИЧЕНИЯ (C*): бюджет, сроки, люди, технические пределы, правила.
- ВОЗРАЖЕНИЯ (O*): несогласие с конкретным предложением или решением.
- ВОПРОСЫ (Q*): открытые вопросы к команде.

Правила:
1. Для каждого сообщения определи тип: idea, requirement, objection, change,
agreement,
   question, answer, chatter. На chatter не создавай операций.
2. Для каждого предложения спроси себя: "ЗАЧЕМ человек этого хочет?" Если
ответ
   совпадает по смыслу с существующей целью — ИСПОЛЬЗУЙ её goal_id. Создавай
новую
   цель, только если ни одна не подходит. Разные способы — не повод для
разных целей.
3. Если сообщение повторяет уже существующую карточку другими словами —
   attach_to_item, а не новая карточка. Если автор уточняет СВОЁ предложение
— update_item.
4. Если меняется или снимается ограничение ("бюджет дали", "срок сдвинули") —
   change_constraint на СУЩЕСТВУЮЩЕЕ ограничение, а не новая карточка.
5. Если сообщение отвечает на открытый вопрос — resolve_question.
6. Если новая карточка ПОХОЖЕ противоречит другой карточке или ограничению,
укажи
   их id в possible_conflict_with. Не решай, настоящий ли это конфликт.
7. Ссылайся ТОЛЬКО на id из карты. Новым карточкам id не придумывай — их
выдаст
   система. Для ссылки на цель, созданную в этом же ответе, используй
new_goal_ref.
8. Формулировки карточек — короткие (до 10 слов), в нейтральном тоне, на
языке чата.
   Не добавляй того, чего человек не говорил.
```

Шаблон пользовательского сообщения:

```
КАРТА ЗАДАЧИ:
Цели:
{goals_compact}            # G1* увеличить продажи на 15% (главная)
Карточки:
{items_compact}            # P1 [маркетолог] персональные предложения | цель
G2 | как: рекомендации
Открытые вопросы:
{questions_compact}
Принятые решения:
{decisions_compact}

КОНТЕКСТ (предыдущие сообщения):
{recent_messages}

НОВЫЕ СООБЩЕНИЯ (разбери их):
{new_messages}             # #14 [Разработчик] Сложную систему рекомендаций
не потянем...
```

Схема ответа:

```
class ExtractOp(BaseModel):
    op: Literal["add_goal", "add_item", "attach_to_item", "update_item",
                "change_constraint", "resolve_question"]
    ref: str | None = None            # временная ссылка для add_goal: "new1"
    kind: ItemKind | None = None
    text: str | None = None
    goal_id: str | None = None        # существующая цель
    new_goal_ref: str | None = None   # или созданная в этом ответе
    means: str | None = None
    item_id: str | None = None        # для attach/update/change/resolve
    is_business_goal: bool = False
    possible_conflict_with: list[str] = []
    source_msg: int

class MessageLabel(BaseModel):
    seq: int
    type: Literal["idea", "requirement", "objection", "change", "agreement",
                  "question", "answer", "chatter"]

class ExtractResult(BaseModel):
    labels: list[MessageLabel]
    ops: list[ExtractOp]
```

Mediator — системный промпт

```
Ты — фасилитатор, который помогает команде прийти к согласованному решению
бизнес-задачи. Тебя вызвали, потому что система заметила ситуацию типа
{trigger}
вокруг карточек {focus_ids}.

Главное правило: НЕ объединяй всё насильно. Сначала разберись, что
происходит:
- same_goal: люди хотят одного и того же, но предлагают разные способы.
  → предложи один общий вариант, который берёт лучшее из способов и
укладывается
    в ограничения.
- complementary: цели разные, но их можно достичь одновременно.
  → предложи вариант, который удовлетворяет обе.
- conflict: цели конкурируют за один ресурс (деньги, время, маржу, людей,
  внимание пользователя), и выигрыш одной стороны — проигрыш другой.
  → НЕ предлагай решение. Назови конфликт прямо, покажи 2–3 варианта
приоритета
    и чем жертвует каждый, спроси, что важнее.
- misunderstanding: люди спорят, но на самом деле говорят о разном или одном
и том же.
  → задай уточняющий вопрос.

Рассуждай по шагам в полях ответа в том порядке, в каком они идут:
1) что на самом деле хочет каждый автор; 2) одна ли это цель;
3) можно ли удовлетворить всех сразу при текущих ограничениях; 4) вердикт;
5) вмешательство.

Если предлагаешь решение:
- каждая карточка из {focus_ids} должна попасть либо в covers (как учтена),
  либо в sacrifices (чем пожертвовали и почему);
- в depends_on перечисли ограничения, без которых решение теряет смысл;
- решение должно быть конкретным и выполнимым при ограничениях.

Текст для чата: до 3 предложений, обращайся к людям по ролям, без
комплиментов
и канцелярита. Структура: что заметил → что предлагаешь или спрашиваешь → чьи
пожелания учтены. Если сомневаешься (confidence < 0.6) — спроси, а не
утверждай.
Ссылайся только на существующие id.
```

В пользовательском сообщении — та же карта задачи (с авторами и ролями) и последние 15 сообщений.

```
class AuthorGoal(BaseModel):
    item_id: str
    real_goal: str                    # что человек на самом деле хочет

class PriorityOption(BaseModel):
    label: str                        # "Маржа важнее"
    sacrifices: str                   # "меньше вернём старых клиентов"

class MediatorResult(BaseModel):
    goals: list[AuthorGoal]
    same_underlying_goal: bool
    same_goal_reason: str
    jointly_satisfiable: bool
    contested_resource: str | None    # за что конкурируют, если конфликт
    relation_type: Literal["same_goal", "complementary", "conflict",
"misunderstanding"]
    confidence: float
    chat_message: str
    action: Literal["propose_decision", "ask_priority", "ask_clarify"]
    decision_text: str | None = None
    rationale: str | None = None
    covers: dict[str, str] = {}
    sacrifices: dict[str, str] = {}
    depends_on: list[str] = []
    question_text: str | None = None
    options: list[PriorityOption] = []
```

Impact — системный промпт

```
У команды изменилось ограничение. Было: "{old}". Стало: "{new}".
Ниже принятые решения, которые на нём держались, с их обоснованиями.
Для каждого решения верни вердикт:
- keep: решение остаётся в силе, объясни почему;
- revise: решение нужно изменить, предложи новую формулировку;
- drop: решение больше не имеет смысла, сформулируй вопрос к команде.
Затем напиши одно сообщение для чата (до 4 предложений): что изменилось
и что придётся пересмотреть. Не пересматривай решения, которых нет в списке.
```

Бэкенд

Один FastAPI-процесс: REST для служебных действий и один WebSocket на комнату для всего живого. **Первым делом, до 11:45, фиксируете с фуллстеком протокол ниже и поднимаете mock-сервер**, который шлёт заготовленные патчи из сценария демо. Тогда фронт не ждёт ML.

REST

Метод	Путь	Что делает
POST	`/rooms`	Создать комнату → `{room_id}`
GET	`/rooms/{id}/state`	Полное состояние + история сообщений
POST	`/rooms/{id}/replay`	Проиграть сценарий из `scenarios/*.json` с паузами (страховка демо)
POST	`/rooms/{id}/reset`	Очистить комнату
GET	`/rooms/{id}/export`	Итоги в markdown: цель, решения с обоснованиями, открытые вопросы
GET	`/debug/llm`	Последние LLM-вызовы: промпт, ответ, задержка, токены (для отладки и питча)

WebSocket `/ws/{room_id}?user=Аня&role=маркетолог`

Направление	type	Payload
клиент → сервер	`message`	`{text}`
клиент → сервер	`vote`	`{decision_id, value: bool, comment?}`
сервер → клиент	`snapshot`	`{state, messages}` — сразу после подключения
сервер → клиент	`message`	`{seq, author, role, text, ts, is_agent}`
сервер → клиент	`patch`	`{version, ops: [...применённые операции с уже выданными id]}`
сервер → клиент	`agent`	`{seq, kind: merge/conflict/impact/clarify, text, decision_id?, question_id?, focus_ids}`
сервер → клиент	`agent_status`	`{status: idle/reading/thinking}` — индикатор «агент думает…»

Правило для фронта: если `patch.version` не равен своей версии + 1, фронт перезапрашивает `/state`. Это закрывает переподключения и потерянные патчи.

Структура репозитория

```
backend/
  app/
    main.py          # FastAPI, WebSocket, REST
    rooms.py         # Room: состояние, очередь, воркер, broadcast
    models.py        # Pydantic: состояние, операции, ответы LLM
    reducer.py       # применение и валидация операций, выдача id
    triggers.py      # trigger policy
    impact.py        # граф зависимостей
    llm.py           # клиент, structured output, ретраи, кэш, лог
    pipeline.py      # extract -> reduce -> trigger -> mediate
    render.py        # состояние -> компактный текст для промпта
  prompts/           # extractor.md, mediator.md, impact.md
  scenarios/         # demo_shop.json + eval-сценарии
  eval/run_eval.py
  data/              # jsonl-журналы, llm_cache/, llm_log.jsonl
frontend/
```

Скелет комнаты и пайплайна

```
# rooms.py
import asyncio, time
from fastapi import WebSocket

DEBOUNCE_S = 1.5

class Room:
    def __init__(self, room_id: str):
        self.state = RoomState(room_id=room_id)
        self.messages: list[Message] = []
        self.sockets: dict[WebSocket, str] = {}
        self.inbox: asyncio.Queue[Message] = asyncio.Queue()
        self.worker = asyncio.create_task(self._run())

    async def broadcast(self, payload: dict):
        dead = []
        for ws in self.sockets:
            try:
                await ws.send_json(payload)
            except Exception:
                dead.append(ws)
        for ws in dead:
            self.sockets.pop(ws, None)

    async def on_message(self, author: str, role: str, text: str):
```

```
        msg = Message(seq=len(self.messages), author=author, role=role,
                      text=text, ts=time.time())
        self.messages.append(msg)
        await self.broadcast({"type": "message", **msg.model_dump()})
        await self.inbox.put(msg)

    async def _run(self):
        while True:
            batch = [await self.inbox.get()]
            await asyncio.sleep(DEBOUNCE_S)          # собираем пачку
            while not self.inbox.empty():
                batch.append(self.inbox.get_nowait())
            try:
                await process_batch(self, batch)     # pipeline.py
            except Exception as e:
                log_error(self.state.room_id, e)     # демо не падает
                await self.broadcast({"type": "agent_status", "status":
"idle"})
```

```
# pipeline.py
async def process_batch(room: Room, batch: list[Message]):
    await room.broadcast({"type": "agent_status", "status": "reading"})

    # 1. Extractor
    extract = await llm.structured(
        model=cfg.FAST_MODEL, system=prompts.extractor,
        user=render.extractor_input(room.state, room.messages, batch),
        schema=ExtractResult)
    applied, signals = reducer.apply_extract(room.state, extract)
    if applied:
        await room.broadcast({"type": "patch", "version": room.state.version,
                              "ops": applied})

    # 2. Нужно ли говорить?
    trig = triggers.pick(room.state, signals, room.messages)
    if trig is None:
        await room.broadcast({"type": "agent_status", "status": "idle"})
        return

    await room.broadcast({"type": "agent_status", "status": "thinking"})

    # 3. Impact: сначала детерминированная подсветка, потом объяснение
    if trig.kind == "IMPACT":
        hit = impact.affected_decisions(room.state, trig.changed_ids)
        ops = reducer.mark_needs_review(room.state, hit, trig.reason)
        await room.broadcast({"type": "patch", "version": room.state.version,
```

```
"ops": ops})
        result = await llm.structured(model=cfg.STRONG_MODEL,
system=prompts.impact,
                                      user=render.impact_input(room.state,
trig, hit),
                                      schema=ImpactResult)
        ops = reducer.apply_impact(room.state, result)
    else:
        result = await llm.structured(model=cfg.STRONG_MODEL,

system=prompts.mediator.format(trigger=trig.kind,

focus_ids=trig.focus_ids),
                                      user=render.mediator_input(room.state,
room.messages),
                                      schema=MediatorResult,

validator=guards.check_mediator(room.state, trig))
        ops = reducer.apply_mediator(room.state, result, trig)

    agent_msg = room.add_agent_message(result.chat_message, trig, ops)
    await room.broadcast({"type": "patch", "version": room.state.version,
"ops": ops})
    await room.broadcast({"type": "agent", **agent_msg})
    await room.broadcast({"type": "agent_status", "status": "idle"})
    triggers.mark_spoken(room.state, trig)
```

`guards.check_mediator` — это правила из раздела «Защита от принудительного слияния». Она возвращает список нарушений. Если список не пустой, `llm.structured` делает ретрай и дописывает нарушения в сообщение.

Модели, задержка и устойчивость

Две модели: быстрая для извлечения и сильная для вмешательств. Обе меняются одной переменной окружения. В 11:00 прогоните один и тот же запрос на 2–3 кандидатах и зафиксируйте выбор по задержке и валидности JSON.

Роль	Класс модели	Примеры	Бюджет задержки
Extractor	Быстрая, дешёвая, умеет structured output	Claude Haiku 5.5, модели класса mini/flash через OpenRouter	до 2 с

Роль	Класс модели	Примеры	Бюджет задержки
Mediator, Impact	Сильная в рассуждении	Claude Sonnet 5.5, модели класса pro	до 6 с
Эмбеддинги (после MVP)	Multilingual	multilingual-e5 локально или embeddings API	до 0,3 с

Если доступен только GigaChat или YandexGPT, заранее проверьте, поддерживают ли они structured output или function calling. Если нет — схема в тексте промпта, извлечение JSON из ответа и ретрай при ошибке валидации.

llm.py: что в нём должно быть

```
# llm.py
import hashlib, json, time
from openai import AsyncOpenAI

client = AsyncOpenAI(base_url=cfg.LLM_BASE_URL, api_key=cfg.LLM_API_KEY)

async def structured(model, system, user, schema, validator=None,
max_retries=1):
    key = hashlib.sha256(f"{model}|{system}|{user}".encode()).hexdigest()
    if cfg.CACHE_MODE in ("replay", "record") and (hit := cache.get(key)):
        return schema.model_validate_json(hit)      # мгновенно и без сети

    messages = [{"role": "system", "content": system},
                {"role": "user", "content": user}]
    for attempt in range(max_retries + 1):
        t0 = time.perf_counter()
        resp = await client.chat.completions.create(
            model=model, messages=messages, temperature=0.1,
timeout=cfg.LLM_TIMEOUT,
            response_format={"type": "json_schema",
                             "json_schema": {"name": schema.__name__,
                                             "schema":
schema.model_json_schema()}})
        raw = resp.choices[0].message.content
        log_call(model, messages, raw, time.perf_counter() - t0, resp.usage,
attempt)
        try:
            result = schema.model_validate_json(raw)
            problems = validator(result) if validator else []
            if not problems:
```

```
                if cfg.CACHE_MODE == "record":
                    cache.put(key, raw)
                return result
        except ValidationError as e:
            problems = [str(e)]
        messages += [{"role": "assistant", "content": raw},
                     {"role": "user", "content": "Исправь ответ: " + ";
".join(problems)}]
    raise LLMFailed(problems)
```

Точные параметры `response_format` зависят от провайдера: сверьтесь с его документацией в первые 15 минут. Некоторые модели надёжнее работают через tool calling с одним инструментом.

Три режима кэша — главная MLOps-страховка

- `off` — всегда живые вызовы (разработка, eval).
- `record` — живые вызовы, удачные ответы пишутся в кэш. Включаете на финальной репетиции демо.
- `replay` — сначала кэш, при промахе живой вызов. На защите сценарий, отрепетированный слово в слово, отрабатывает мгновенно и так же, как на репетиции. А если жюри попросит написать своё сообщение, оно уйдёт в живую модель.

Ключ кэша — хэш всего промпта вместе с состоянием. Поэтому тексты сообщений на защите должны совпадать с репетицией до символа. Решение: у каждого игрока есть кнопка «вставить следующую реплику» (скрытая демо-панель, см. раздел рисков). Debounce на защите поставьте таким же, как на репетиции, и шлите реплики с паузой больше debounce, иначе пачки соберутся по-другому.

Деградация при отказах

- Таймаут или ошибка Extractor → сообщение остаётся в чате без карточек. Следующая пачка захватит его в контекст.
- Ошибка Mediator → агент молчит, триггер не помечается как отработанный и сработает ещё раз.
- Основной провайдер лежит → запасной ключ или модель в конфиге (`FALLBACK_MODEL`).

Наблюдаемость

Каждый вызов пишется в `data/llm_log.jsonl`: модель, промпт, ответ, задержка, токены, номер попытки, версия промпта (git hash). Страница `/debug/llm` показывает последние 20 вызовов. Если пара минут найдётся, можно подключить Langfuse. Но и jsonl хватит, чтобы показать жюри пайплайн вживую.

Оценка качества (eval)

Набор из 15–20 мини-сценариев с разметкой и скрипт, который прогоняет их после каждой правки промпта. Ведёт тестер с 11:00. Цифры отсюда идут на слайд про ИИ — почти никто на вайбхаке не покажет метрики качества агента.

Формат сценария

```
{
  "id": "conflict_discount_margin",
  "messages": [
    {"author": "Олег", "role": "руководитель", "text": "Маржа не ниже 30%,
это не обсуждается"},
    {"author": "Аня", "role": "маркетолог", "text": "Давайте разошлём скидку
25% по всей базе"}
  ],
  "expect": {
    "relation": "conflict",
    "agent_speaks": true,
    "action": "ask_priority",
    "must_not": ["propose_decision"]
  }
}
```

Категории сценариев

Категория	Сколько	Пример	Ожидание
Одна цель, разные способы	4	«Добавим отзывы на карточку» + «покажем рейтинг продавца» (оба про доверие)	same_goal, propose_decision

Категория	Сколько	Пример	Ожидание
Настоящий конфликт	4	Скидка 25% vs маржа 30%; «запуск за неделю» vs «без тестов не выкатываем»	conflict, ask_priority, никакого propose_decision
Разные цели, можно совместить	3	«Нужны данные для CRM» + «уберём обязательную регистрацию»	complementary, компромисс
Недопонимание / дубль	3	«Нужен лендинг» vs «нет, нужна простая главная»	misunderstanding или duplicate, ask_clarify
Шум	2	«ок», «го обедать», «ахах»	ноль операций, агент молчит
Смена ограничения	2	Принятое решение + «срок сократили вдвое»	change_constraint, нужные решения в needs_review
Полный сценарий демо	1	Таблица из раздела «Сценарий демо»	все три вмешательства в нужных местах

Метрики

Метрика	Как считаем	Цель
Forced merge rate	Доля конфликтов, на которые агент предложил решение	0 — главная метрика продукта
Relation accuracy	Совпадение `relation_type` с разметкой	как можно выше
False trigger rate	Агент заговорил там, где `agent_speaks: false`	близко к 0
Dedup accuracy	Повтор привязан к существующей карточке, а не создан заново	как можно выше
JSON validity	Доля ответов, прошедших валидацию с первой попытки	близко к 100%

Метрика	Как считаем	Цель
Latency p50 / p95	По логу вызовов, отдельно Extractor и Mediator	в бюджете из раздела «Модели»

`python eval/run_eval.py --prompts-rev HEAD` прогоняет все сценарии через настоящий `process_batch` (без WebSocket), печатает таблицу метрик и список проваленных сценариев с ответом модели. Результат пишется в `eval/results.csv` с версией промпта, чтобы на слайде показать рост качества от версии к версии.

Рассинхрон между разметкой и ответом иногда означает, что ошиблась разметка. Тестер разбирает каждый провал вместе с тобой и помечает: ошибка модели или спорный случай. На питч — только реальные цифры последнего прогона.

Роли и план на день

Рабочего времени 5,5 часа: 11:00–13:00 и 14:00–17:30. Код пишут двое, поэтому тестер, ПМ и дизайнер забирают всё, что не код: сценарии, промпт-тесты, дизайн, питч. Так же закрывается критерий «Работа в команде» (10%).

Кто	Отвечает за	Артефакты
Ты (ML + бэк)	`models.py`, `reducer.py`, `llm.py`, `pipeline.py`, `triggers.py`, `impact.py`, промпты, кэш	Работающий пайплайн, версии промптов
Фуллстек	Фронт (чат, доска на React Flow, карточки голосования, журнал решений), `main.py` + `rooms.py`, mock-сервер, деплой	UI, WebSocket-слой
Тестер	Eval-сценарии и их разметка, прогоны после правок промпта, ручное тестирование UI с нескольких вкладок	`scenarios/*.json`, `eval/results.csv`, список багов

Кто	Отвечает за	Артефакты
ПМ	Сценарий демо дословно, тайминг, заморозка объёма, слайды и питч, запись процесса работы с ИИ для слайда	`scenarios/demo_shop.json`, питч, таймер
Дизайнер	Макет экрана до 12:00, цвета типов карточек и рёбер, состояния (новая, в решении, needs_review), логотип, слайды	Макет, дизайн-токены для фронта

Таймлайн

Время	Ты	Фуллстек	Тестер	ПМ	Дизайнер
11:00–11:45	`models.py`, выбор моделей, `llm.py` с structured output	Каркас фронта + бэка, протокол WS, mock-сервер	Первые 8 eval-сценариев	Демо-сценарий дословно в JSON	Макет экрана
11:45–13:00	Extractor + reducer, первый прогон на сценарии	Чат + доска на mock-патчах	Ещё 8 сценариев, скелет `run_eval.py` с тобой	Структура питча	Состояния карточек, дизайн-токены
14:00–15:30	Trigger policy + Mediator + guards	Карточки голосования, журнал решений, индикатор «агент думает»	Eval после каждой правки промпта	Слайды черновиком	Слайды, помощь фронту со стилями
15:30–16:30	Impact-анализ, интеграция с фронтом	Подсветка needs_review, анимации доски	Прогон демо с 4 вкладок, баги	Репетиция реплик	Финальные слайды
16:30–17:15	Тюнинг промптов по провалам eval	Демо-панель, полировка	Финальный eval → цифры на слайд	Запись запасного видео	—
17:15–17:30	**Заморозка кода**. Репетиция в `CACHE_MODE=record`				

Контрольные точки

Если точка не пройдена вовремя, ПМ режет объём, а не двигает срок.

- [ ] 11:45 — протокол WS зафиксирован, mock-сервер отдаёт патчи, модели выбраны
- [ ] 13:00 — Extractor на сценарии демо строит правильные карточки и цели; фронт рисует доску из патчей
- [ ] 15:30 — **сценарий демо проходит от начала до конца на живых моделях**, пусть некрасиво
- [ ] 16:30 — impact-анализ работает, eval без forced merge
- [ ] 17:15 — запасное видео записано, кэш репетиции сохранён, код заморожен

Порядок срезания объёма: сначала NUDGE и @упоминания, потом экспорт, потом анимации. Impact-анализ и различение конфликта и общей цели не режем никогда — это и есть продукт.

Риски, страховка и питч

Риск	Что делаем
Модель на защите отвечает иначе, чем на репетиции	`CACHE_MODE=replay` + реплики через демо-панель (тексты совпадают до символа)
Нет интернета или Wi-Fi режет WebSocket	Бэк локально на одном ноутбуке, раздача с телефона; в крайнем случае все 4 вкладки на одной машине
Агент тормозит (пауза в демо)	Индикатор «агент читает / думает», карточки появляются до вмешательства, replay-кэш
Агент склеил конфликт	Guards в коде запрещают `propose_decision` при `jointly_satisfiable = false`; eval ловит регрессии
Агент лезет на каждое сообщение	Trigger policy на правилах + cooldown; пороги в конфиге
Фронт не успевает	React Flow → простые колонки «Цели / Предложения / Ограничения / Решения / Вопросы», связи — бейджами на карточках

Риск	Что делаем
Ничего не работает на защите	Запасное видео полного сценария, записанное в 17:00

Демо-панель — скрытая кнопка (например, по `?demo=1`). Она подставляет следующую реплику игрока из `demo_shop.json` в поле ввода. Игрок жмёт Enter сам: для жюри это живая переписка, а для кэша — идентичный текст. Честно скажите жюри, что реплики заготовлены, и предложите кому-то из них написать своё — это уйдёт в живую модель и выглядит сильнее любого слайда.

Питч на 3–4 минуты

1. **Проблема (15 с).** «Десятки сообщений — и никакого решения». Один скриншот типичного чата.
2. **Живое демо (2 мин).** Четверо спорят, доска собирается. Акцент на паре сцен «одна цель → объединил» и «конфликт → не склеил, спросил», финал — краснеющие решения.
3. **Как устроено (40 с).** Диаграмма пайплайна: две модели, операции вместо переписывания состояния, guards в коде, детерминированный граф зависимостей.
4. **Качество (20 с).** Реальные цифры eval: точность различения, forced merge, задержка.
5. **Как работали с ИИ (20 с)** — критерий «Использование ИИ»: ИИ внутри продукта (двухступенчатый пайплайн) и ИИ в процессе (кто какими агентами писал код, как генерили eval-сценарии и дизайн). ПМ ведёт заметки об этом весь день.
6. **Финал (15 с).** Журнал решений как память команды, ссылка на открытый репозиторий, куда можно встроить: Telegram-группы, Slack, созвоны через транскрипцию.

Вопросы, к которым стоит подготовиться: «Чем это лучше саммари от ChatGPT?» (ответ: саммари сглаживает конфликты, а мы их явно вытаскиваем и привязываем решения к ограничениям); «А если агент ошибётся?» (ответ: он ничего не решает сам — каждое решение проходит голосование, при сомнении он спрашивает); «Как это масштабируется на 300 сообщений?» (ответ: в промпт идёт компактное состояние, а не история; дальше — эмбеддинги для поиска дублей).